Project 3: Summary Report

Data Ingestion from S3 to RDS with Fallback to AWS Glue using a Dockerized Python Application

Prepared by: Shraavi Ramteke

1. Objective

Build a containerized Python application that reads a CSV file from an Amazon S3 bucket and loads it into an Amazon RDS (MySQL) table. If the load into RDS fails, the application must automatically fall back to AWS Glue and register the same data as a table in the Glue Data Catalog, so the data stays queryable.

2. Data Flow

1. The Docker container starts and runs app.py. All settings (bucket, database, credentials) arrive as environment variables.
2. The script reads data/students.csv from the S3 bucket with boto3 and parses it into a pandas DataFrame (5 rows in this project).
3. It connects to the RDS MySQL database through SQLAlchemy and PyMySQL and writes the DataFrame to the students table.
4. If the RDS write raises any error (for example wrong password or unreachable database), the error is logged and the fallback starts.
5. The fallback creates the Glue database and an external CSV table whose location is the same S3 folder, so the data can be queried from the Glue Data Catalog.

3. AWS Services Used

Service	Resource	Purpose
Amazon S3	Bucket shraavi-project3-data-2026, object data/students.csv	Stores the source CSV file (and is the data location for the Glue table).
Amazon RDS	MySQL instance project3-db (db.t4g.micro), database studentdb	Primary destination. The data is stored in the students table.
AWS Glue	Database student_glue_db, table students_table	Fallback destination. External table over the CSV in S3.
AWS IAM	Dedicated IAM user with access to S3, RDS and Glue	Provides the access keys the container uses to call the AWS APIs.

4. Docker Setup

- **Base image:** python:3.9-slim, working directory `/app`.
- **Dependencies** (requirements.txt): boto3, numpy 1.26.4, pandas 2.0.3, sqlalchemy 2.0.30, pymysql, cryptography. They are installed before the code is copied, so Docker can cache that layer.
- **Entry point:** `CMD ["python", "app.py"]`.
- **Build:** `docker build -t s3-rds-glue-app .`
- **Run:** `docker run --rm -e ... s3-rds-glue-app` with AWS credentials, S3, RDS and Glue settings passed as -e environment variables. No secrets are stored in the image or the code.

5. Results

- **RDS path:** the container read 5 rows from S3 and logged "SUCCESS: data pushed to RDS table students". A MySQL query (SELECT * FROM students) shows all 5 records in the correct columns (id, name, course, city).

- **Fallback path:** with a deliberately wrong RDS password, the RDS write failed with "Access denied (1045)". The application logged "Falling back to AWS Glue..." and then "Glue table created: student_glue_db.students_table". The table is visible in the Glue Data Catalog with classification CSV and location s3://shraavi-project3-data-2026/data/.

6. Challenges and Solutions

Challenge	Solution
Container crashed on import pandas: ValueError “numpy.dtype size changed, may indicate binary incompatibility”	pip had pulled in a newer NumPy (2.x) that is not binary-compatible with pandas 2.0.3. Pinned numpy==1.26.4 in requirements.txt and rebuilt the image; pandas then imported correctly and the script ran.
Could not connect to RDS (ERROR 2003 on port 3306)	Checked that the instance was Available and that the security group inbound rule for port 3306 allowed my current IP address.

7. Conclusion

The application meets the requirement: it ingests data from S3 into RDS and, when RDS is not usable, keeps the data available through AWS Glue without manual intervention. Possible improvements are moving to a Python 3.10+ base image (boto3 now warns about Python 3.9), reading credentials from an IAM role or AWS Secrets Manager instead of environment variables, and making the RDS load idempotent so re-runs do not create duplicate rows.